

ECUE 423 - Développement WEB pour la Santé

JQuery

- Bibliothèque Javascript pour simplifier la manipulation du DOM.
- Avantages :
 - Gain de temps de développement.
 - Validité de notre code.
- En effet, les créateurs de jQuery s'occupent de la validité parfaite de leur code pour l'ensemble des navigateurs les plus utilisés.
- jQuery assure la compatibilité inter-navigateurs et interversions de notre code.

- Site officiel :
 - <https://jquery.com/>
- Sur la page de téléchargement, deux types de fichiers vont vous être proposés : des fichiers compressés et des fichiers non compressés.
- La copie de jQuery compressée ne contient aucun espace entre les codes.
 - Le but est de réduire le poids total de la librairie afin d'améliorer les performances de votre site. C'est la version que nous placerons généralement sur notre site « live » (en production).
- Cependant, pour travailler, je vous conseille plutôt la version non compressée, qui est une version lisible de la librairie (avec notamment des commentaires, une bonne indentation, etc.).
 - Cela vous permettra d'étudier plus facilement, si vous le souhaitez, les scripts composant la librairie jQuery.
- Vous pouvez aussi inclure avec un CDN (Google par exemple) :

```
<head>  
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>  
</head>
```

La syntaxe du jQuery

- Une fois la librairie jQuery incluse dans nos scripts, jQuery va créer deux objets jQuery et \$ qui vont être des propriétés de l'objet global window.
- Nous allons pouvoir accéder aux différentes propriétés et méthodes de la librairie jQuery en utilisant jQuery() ou \$().
 - Ce sont des alias.
 - La plupart des développeurs utilise plutôt la propriété \$ puisqu'elle est plus rapide à écrire que jQuery.

La syntaxe du jQuery

- Le jQuery est ensuite un langage de “queries” JavaScript c’est-à-dire un langage de requêtes JavaScript.
- On utilise des sélecteurs CSS avec la méthode `$()` (ou dans la méthode `jquery()`).
- Pour sélectionner tous les paragraphes d’une page, nous écrirons donc `$("p")` tandis que pour ne sélectionner que l’élément possédant un attribut `id="p1"`, on écrira `$("#p1")`.
- Cela marche aussi pour les sélecteurs CSS complexes ou composés.
- `$(selecteur)` retourne un objet jQuery.
 - Les codes jQuery vont donc ressembler à cela :
 - `$(selecteur).action()`
 - Ici, la partie `$(selecteur)` renvoie un objet auquel on applique la méthode `action()`.

La syntaxe du jQuery

7

cours.html ×

cours.html > html

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <script src="jquery3.7.1.min.js"></script>
7   </head>
8   <body>
9     <h1>Cours jQuery</h1>
10    <p id="p1">Cliquez moi</p>
11    <script src="cours.js"></script>
12  </body>
13 </html>
```

JS cours.js ×

JS cours.js

```
1 $("p").hide();
```

Sélectionne tous les éléments p de notre page avec \$("p").

Ce code renvoie un objet jQuery auquel on applique la méthode jQuery hide() qui va se charger de cacher tous les paragraphes de notre page.

La syntaxe du jQuery

- Pour manipuler cette structure HTML, il faut qu'elle existe.
- Il faudra donc s'assurer que le navigateur a fini de charger la structure HTML de notre page avant d'effectuer une quelconque opération en jQuery.
- jQuery met à notre disposition l'événement **ready**.
 - Cet événement va nous permettre d'exécuter notre code jQuery dès que le document est prêt ("ready") à être manipulé.
- On va pouvoir utiliser cet événement comme cela.
 - Attention :
 - `$(document).ready() = $()`

```
JS cours.js × ✓ 📖 📄 🖨
JS cours.js > ...
1  /*
2  $("p").hide();
3  */
4
5  /*Dès que le document est prêt à être manipulé, on
6  à l'intérieur de notre fonction anonyme*/
7  $(document).ready(function(){
8      $("p").hide();
9  })
10
11 // $(document).ready() = $()
12 /*
13 $(function(){
14     $("p").hide();
15 })*/*
```


La sélection d'éléments en jQuery

- jQuery utilise des sélecteurs CSS.
- De plus, jQuery met également à notre disposition des “pseudo-sélecteurs” qui vont nous permettre de sélectionner précisément et simplement certains éléments spécifiques aux formulaires HTML.
- On va notamment pouvoir sélectionner des éléments input en fonction de la valeur de leur attribut type grâce aux pseudo-sélecteurs jQuery
 - `:text`, `:password`, `:checkbox`, `:radio`, `:submit`, `:file`,
`:image`, `:button`, `:reset`

La sélection d'éléments en jQuery

- jQuery nous permet également de sélectionner certains éléments de formulaire en fonction de leur état.
 - `:input` pour cibler tous les éléments `input`, `textarea`, `select` et `button` ;
 - `:checked` pour cibler les éléments cochés ou sélectionnés pour des `input type="checkbox"`, `input type="radio"` et des éléments de liste `select` ;
 - `:selected` pour cibler les éléments `option` sélectionnés dans une liste `select` ;
 - `:disabled` pour cibler les éléments `input` qui possèdent un attribut `disabled` ;
 - `:enabled` pour cibler tous les éléments qui ne possèdent pas d'attribut `disabled`.
- Enfin, jQuery nous fournit également quelques méthodes qui vont nous permettre d'affiner nos sélections. On peut notamment ici citer les méthodes `has()`, `filter()`, `not()`, `first()`, `last()` et `eq()`.

La méthode jQuery has()

- La méthode jQuery has() va nous permettre d'affiner une sélection en ne sélectionnant que les éléments d'une sélection initiale qui possèdent des éléments enfants correspondant au sélecteur passé en argument de cette fonction.
- Cette méthode renvoie un objet jQuery qui contient la sous-sélection.
- On va ainsi par exemple pouvoir sélectionner uniquement les éléments p de notre page qui contiennent un élément span.
- Ici, j'utilise la méthode css() pour modifier la couleur des éléments sélectionnés par has() afin de les mettre en relief.

La méthode jQuery has()

12

```
<> cours2.html x
<> cours2.html > html
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours jQuery</title>
5      <meta charset="utf-8">
6      <script src="jquery3.7.1.min.js"></script>
7    </head>
8    <body>
9      <h1>Cours jQuery</h1>
10     <p id="p1">Un paragraphe <span>écrit en HTML</span></p>
11     <span>Un élément span non contenu dans un élément p</span>
12     <p id="p2">Un deuxième paragraphe</p>
13     <p>Un <span>troisième</span> paragraphe</p>
14     <script src="cours2.js"></script>
15   </body>
16 </html>
```

```
JS cours2.js x
JS cours2.js > ...
1  /*On cible les éléments p qui contiennent un élément span et on
2  leur applique une couleur orange*/
3  $(document).ready(function(){
4    let pspan = $("p").has("span").css("color", "orange");
5  });
6
```

← → ↻ ⓘ Fichier /Users/thiago/Library/

Cours jQuery

Un paragraphe écrit en HTML

Un élément span non contenu dans un élément p

Un deuxième paragraphe

Un troisième paragraphe

La méthode jQuery filter()

- La méthode jQuery filter() va nous permettre d'affiner une sélection en ne sélectionnant que les éléments qui satisfont au sélecteur passé en argument de cette fonction.
- On va ainsi par exemple pouvoir ne sélectionner que les éléments span d'une page qui possèdent un certain attribut class.

La méthode jQuery filter()

14

```
<> cours3.html X
cours3.html > ...
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <script src="jquery3.7.1.min.js"></script>
7     <style>
8       .souligne{
9         text-decoration: underline;
10      }
11    </style>
12  </head>
13  <body>
14    <h1 class="souligne">Cours jQuery</h1>
15    <p id="p1">Un paragraphe <span class="souligne">écrit en HTML</span></p>
16    <span>Un élément span non contenu dans un élément p</span>
17    <p id="p2">Un deuxième paragraphe</p>
18    <p>Un <span class="souligne">troisième</span> paragraphe</p>
19    <script src="cours3.js"></script>
20  </body>
21 </html>
```

```
JS cours3.js X
JS cours3.js > ...
1 /*On ne sélectionne que les éléments span qui possèdent un attribut
2 class="souligne"*/
3 $(document).ready(function(){
4   $("span").filter(".souligne").css("color", "green");
5 });
```

← → ↻ ⓘ Fichier /Users/thiago/Library

Cours jQuery

Un paragraphe écrit en HTML

Un élément span non contenu dans un élément p

Un deuxième paragraphe

Un troisième paragraphe

La méthode jQuery not()

- La méthode not() va nous permettre de soustraire d'une sélection de départ des éléments qui répondent à un certain critère.
- On va ainsi par exemple pouvoir supprimer d'une sélection les éléments span possédant un certain attribut class et appliquer des styles à tous les span sauf à ceux possédant cet attribut class.

La méthode jQuery not()

16

```
<> cours4.html ×
<> cours4.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours jQuery</title>
5      <meta charset="utf-8">
6      <script src="jquery3.7.1.min.js"></script>
7      <style>
8        .souligne{
9          text-decoration: underline;
10       }
11     </style>
12   </head>
13   <body>
14     <h1 class="souligne">Cours jQuery</h1>
15     <p id="p1">Un paragraphe <span class="souligne">écrit en HTML</span></p>
16     <span>Un élément span non contenu dans un élément p</span>
17     <p id="p2">Un deuxième paragraphe</p>
18     <p>Un <span class="souligne">troisième</span> paragraphe</p>
19     <script src="cours4.js"></script>
20   </body>
21 </html>
```

```
JS cours4.js ×
JS cours4.js > ...
1  /*On ne sélectionne que les éléments span qui NE possèdent PAS
2  d'attribut class="souligne"*/
3  $(document).ready(function(){
4    $("span").not(".souligne").css("color", "green");
5  });
6
```

← → ↻ ⓘ Fichier /Users/thiago/Librar

Cours jQuery

Un paragraphe écrit en HTML

Un élément span non contenu dans un élément p

Un deuxième paragraphe

Un troisième paragraphe

Les méthodes jQuery first(), last() et eq()

- Les méthodes first() et last() vont nous permettre d'affiner une sélection en ne sélectionnant respectivement que le premier et le dernier élément de la sélection de départ.
- La méthode eq() va elle nous permettre de ne sélectionner qu'un élément dans une sélection de départ en fonction de sa place dans cette sélection. Elle est donc plus flexible que first() ou last().
- Pour sélectionner un élément en fonction de sa position dans la collection de départ, nous allons devoir fournir un index (ou indice) à eq(). Ici, il faut simplement retenir que le premier élément de la collection de départ possède l'indice 0, le deuxième l'indice 1, le troisième l'indice 2, etc.
- Pour sélectionner le deuxième élément de notre sélection de base, on écrira donc eq(1).
- Vous pouvez également noter qu'on va pouvoir passer des indices négatifs à eq(). Dans ce cas, la sélection se fera en partant de la fin : eq(-1) permettra de sélectionner le dernier élément de notre sélection de départ, eq(-2) l'avant dernier élément, etc.

Les méthodes jQuery first(), last() et eq()

18

cours5.html x

✓ 📄 🖨 📄 ...

cours5.html > ...

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <script src="jquery3.7.1.min.js"></script>
7     <style>
8       .souligne{
9         text-decoration: underline;
10      }
11    </style>
12  </head>
13  <body>
14    <h1 class="souligne">Cours jQuery</h1>
15    <p id="p1">Un paragraphe <span class="souligne">écrit en HTML</span></p>
16    <span>Un élément span non contenu dans un élément p</span>
17    <p id="p2">Un deuxième paragraphe</p>
18    <p>Un <span class="souligne">troisième</span> paragraphe</p>
19    <script src="cours5.js"></script>
20  </body>
21 </html>
```

JS cours5.js x

JS cours5.js > ...

```
1 $(document).ready(function(){
2   $("span").first().css("color", "green");
3   $("span").eq(1).css("color", "blue");
4   $("span").last().css("color", "orange");
5 });
6
```

← → ↻ ⓘ Fichier /Users/thiago/Library/Clou

Cours jQuery

Un paragraphe écrit en HTML

Un élément span non contenu dans un élément p

Un deuxième paragraphe

Un troisième paragraphe

- Les méthodes `parent()`, `parents()`, `parentsUntil()` et `closest()` vont nous permettre d'accéder aux parent et ancêtres d'un élément ou d'une collection d'éléments.
- La méthode `parent()` nous permet d'accéder à l'élément parent (c'est-à-dire à l'ancêtre direct) d'un premier élément sélectionné ou à chaque parent d'un ensemble d'éléments sélectionnés. On va pouvoir lui passer un sélecteur en argument qui permet de filtrer la sélection et de ne retourner le parent d'un élément que si celui-ci correspond au sélecteur.

Accéder à l'élément parent ou aux ancêtres d'un élément jQuery

20

```
< coursDOM1.html x
< coursDOM1.html > html > body > div.conteneur > div > p > span
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <script src="jquery3.7.1.min.js"></script>
7     <style>
8       .souligne{
9         text-decoration: underline;
10      }
11    </style>
12  </head>
13  <body>
14    <div class="conteneur">
15      <h1>Cours jQuery</h1>
16      <div>
17        <p>Un paragraphe HTML</p>
18      </div>
19      <div>
20        <p>Quelques méthodes jQuery permettant de <span>
21          remonter</span> le DOM :</p>
22        <ul id="parents">
23          <li>parent()</li>
24          <li>parents()</li>
25          <li>parentsUntil()</li>
26          <li>closest()</li>
27        </ul>
28      </div>
29      <p>Quelques méthodes permettant de <span>descendre</span>
30        le DOM :</p>
31      <ul id="enfants">
32        <li>children()</li>
33        <li>find()</li>
34      </ul>
35    </div>
36  </div>
37  <script src="coursDOM1.js"></script>
38 </body>
39 </html>
```

```
JS coursDOM1.js x
JS coursDOM1.js > ...
1 $(document).ready(function(){
2   /*Cible l'élément parent direct de chaque élément li et ajoute
3     une bordure autour*/
4   $("li").parent().css("border", "1px solid black");
5
6   /*Cible l'élément parent direct de chaque élément li uniquement si
7     il possède un id="parents" et change la couleur de fond en bleu*/
8   $("li").parent("#parents").css("background-color", "lightBlue");
9 });
```

← → ↺ ⓘ Fichier /Users/thiago/Libr... ☆ 🔔

Cours jQuery

Un paragraphe HTML

Quelques méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Quelques méthodes permettant de descendre le DOM :

- children()
- find()

Accéder à l'élément parent ou aux ancêtres d'un élément jQuery

21

- La méthode `parents()` permet d'accéder à tous les ancêtres d'un élément ou d'un groupe d'éléments sélectionnés. Cette méthode va fonctionner de la même manière que `parent()` et va également accepter un sélecteur en argument qui va servir de filtre.

Accéder à l'élément parent ou aux ancêtres d'un élément jQuery

22

```
<> coursDOM2.html > ...
<> coursDOM2.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head> ...
12 </head>
13 <body>
14   <div class="conteneur">
15     <h1>Cours jQuery</h1>
16     <div>
17       <p>Un paragraphe HTML</p>
18     </div>
19     <div>
20       <p>Méthodes jQuery permettant de <span>remonter</span> le DOM :</p>
21       <ul id="parents">
22         <li>parent()</li>
23         <li>parents()</li>
24         <li>parentsUntil()</li>
25         <li>closest()</li>
26       </ul>
27
28       <p>Méthodes permettant de <span>descendre</span> le DOM :</p>
29       <ul id="enfants">
30         <li>children()</li>
31         <li>find()</li>
32       </ul>
33     </div>
34   </div>
35   <footer>
36     <ul>
37       <li>Élément de pied de page n°1</li>
38       <li>Élément de pied de page n°2</li>
39     </ul>
40   </footer>
41   <script src="coursDOM2.js"></script>
42 </body>
43 </html>

JS coursDOM2.js > ...
JS coursDOM2.js > ...
1  $(document).ready(function(){
2    /*Sélectionne tous les ancêtres des éléments ul et
3     ajoute une bordure noire d'1px autour de
4     chacun d'entre eux*/
5    $("ul").parents().css("border", "1px solid black");
6
7    /*Ne cible que les ancêtres des éléments ul qui sont
8     des div et change leur couleur de fond en bleu*/
9    $("ul").parents("div").css("background-color", "lightBlue");
10  });
11
```

Accéder à l'élément parent ou aux ancêtres d'un élément jQuery

23

```
<? coursDOM2.html > ...
<? coursDOM2.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head> ...
12 </head>
13 <body>
14   <div class="conteneur">
15     <h1>Cours jQuery</h1>
16     <div>
17       <p>Un paragraphe HTML</p>
18     </div>
19     <div>
20       <p>Méthodes jQuery permettant de <span>remonter</span> le DOM :</p>
21       <ul id="parents">
22         <li>parent()</li>
23         <li>parents()</li>
24         <li>parentsUntil()</li>
25         <li>closest()</li>
26       </ul>
27       <p>Méthodes permettant de <span>descendre</span> le DOM :</p>
28       <ul id="enfants">
29         <li>children()</li>
30         <li>find()</li>
31       </ul>
32     </div>
33   </div>
34   <footer>
35     <ul>
36       <li>Elément de pied de page n°1</li>
37       <li>Elément de pied de page n°2</li>
38     </ul>
39   </footer>
40   <script src="coursDOM2.js"></script>
41 </body>
42 </html>
```

Fichier /Users/thiago/Library/Cloud... ☆

1
2
3
4
5
6
7
8
9
10
11

Cours jQuery

Un paragraphe HTML

Méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Méthodes permettant de descendre le DOM :

- children()
- find()

- Elément de pied de page n°1
- Elément de pied de page n°2

- La méthode `parentsUntil()` permet également d'accéder aux ancêtres d'un élément ou d'un ensemble d'éléments jusqu'à l'ancêtre correspondant au sélecteur qu'on va passer en argument de cette méthode.
- Attention ici : l'ancêtre correspondant au sélecteur ne sera pas inclus dans les résultats. Si aucun ancêtre ne correspond au sélecteur passé ou si aucun argument n'est fourni à `parentsUntil()`, tous les ancêtres des éléments seront sélectionnés.
- On va pouvoir passer un sélecteur en second argument de `parentsUntil()` qui va servir de filtre (les ancêtres au milieu ne correspondant pas au filtre seront ignorés).

coursDOM3.html

coursDOM3.html > ...

```

1  <!DOCTYPE html>
2  <html>
3      <head>
4          <title>Cours jQuery</title>
5          <meta charset="utf-8">
6          <script src="jquery3.7.1.min.js"></script>
7          <style>
8              body, div{padding: 10px;}
9          </style>
10     </head>
11     <body>
12         <div class="conteneur">
13             <h1>Cours jQuery</h1>
14             <div>
15                 <p>Un paragraphe HTML</p>
16             </div>
17             <div>
18                 <p>Méthodes jQuery permettant de <span>remonter</span> le DOM :</p>
19                 <ul id="parents">
20                     <li>parent()</li>
21                     <li>parents()</li>
22                     <li>parentsUntil()</li>
23                     <li>closest()</li>
24                 </ul>
25
26                 <p>Méthodes permettant de <span>descendre</span> le DOM :</p>
27                 <ul id="enfants">
28                     <li>children()</li>
29                     <li>find()</li>
30                 </ul>
31             </div>
32         </div>
33         <footer>
34             <ul>
35                 <li>Elément de pied de page n°1</li>
36                 <li>Elément de pied de page n°2</li>
37             </ul>
38         </footer>
39         <script src="coursDOM3.js"></script>
40     </body>
41 </html>

```

JS coursDOM3.js

JS coursDOM3.js > ...

```

1  $(document).ready(function(){
2      /*Sélectionne tous les ancêtres de l'élément qui
3      possède l'id="enfants" jusqu'au premier ancêtre
4      possédant une class="conteneur" sans l'inclure*/
5      $("#enfants").parentsUntil(".conteneur").css(
6          "border", "1px solid black");
7  });
8

```

Cours jQuery

Un paragraphe HTML

Méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Méthodes permettant de descendre le DOM :

- children()
- find()

- Elément de pied de page n°1
- Elément de pied de page n°2

Accéder à l'élément parent ou aux ancêtres d'un élément jQuery

26

- La méthode `parents()` permet d'accéder à tous les ancêtres d'un élément ou d'un groupe d'éléments sélectionnés.
- Cette méthode va fonctionner de la même manière que `parent()` et va également accepter un sélecteur en argument qui va servir de filtre.

HTML

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <style>
7       body, div{padding: 10px;}
8     </style>
9   </head>
10  <body>
11    <div class="conteneur">
12      <h1>Cours jQuery</h1>
13      <div>
14        <p>Un paragraphe HTML</p>
15      </div>
16      <div>
17        <p>Méthodes jQuery permettant de <span>remonter</span> le DOM :</p>
18        <ul id="parents">
19          <li>parent()</li>
20          <li>parents()</li>
21          <li>parentsUntil()</li>
22          <li>closest()</li>
23        </ul>
24
25        <p>Méthodes permettant de <span>descendre</span> le DOM :</p>
26        <ul id="enfants">
27          <li>children()</li>
28          <li>find()</li>
29        </ul>
30      </div>
31    </div>
32    <footer>
33      <ul>
34        <li>Elément de pied de page n°1</li>
35        <li>Elément de pied de page n°2</li>
36      </ul>
37    </footer>
38  </body>
39 </html>

```

JS

```

1 $(document).ready(function(){
2   /*Sélectionne tous les ancêtres des éléments ul et ajoute une bordure
3     noire d'1px autour de chacun d'entre eux*/
4   $("ul").parents().css("border", "1px solid black");
5
6   /*Ne cible que les ancêtres des éléments ul qui sont des div et change
7     leur couleur de fond en bleu*/
8   $("ul").parents("div").css("background-color", "lightBlue");
9 });
10

```

Cours jQuery

Un paragraphe HTML

Méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Méthodes permettant de descendre le DOM :

- children()
- find()

- Elément de pied de page n°1
- Elément de pied de page n°2

- La méthode `parentsUntil()` permet également d'accéder aux ancêtres d'un élément ou d'un ensemble d'éléments jusqu'à l'ancêtre correspondant au sélecteur qu'on va passer en argument de cette méthode.
- Attention ici : l'ancêtre correspondant au sélecteur ne sera pas inclus dans les résultats. Si aucun ancêtre ne correspond au sélecteur passé ou si aucun argument n'est fourni à `parentsUntil()`, tous les ancêtres des éléments seront sélectionnés.
- On va pouvoir passer un sélecteur en second argument de `parentsUntil()` qui va servir de filtre (les ancêtres au milieu ne correspondant pas au filtre seront ignorés).

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <style>
7       body, div{padding: 10px;}
8     </style>
9   </head>
10  <body>
11    <div class="conteneur">
12      <h1>Cours jQuery</h1>
13      <div>
14        <p>Un paragraphe HTML</p>
15      </div>
16      <div>
17        <p>Méthodes jQuery permettant de <span>remonter</span> le
DOM :</p>
18        <ul id="parents">
19          <li>parent()</li>
20          <li>parents()</li>
21          <li>parentsUntil()</li>
22          <li>closest()</li>
23        </ul>
24
25        <p>Méthodes permettant de <span>descendre</span> le DOM :
</p>
26        <ul id="enfants">
27          <li>children()</li>
28          <li>find()</li>
29        </ul>
30      </div>
31    </div>
32    <footer>
33      <ul>
34        <li>Elément de pied de page n°1</li>
35        <li>Elément de pied de page n°2</li>
36      </ul>
37    </footer>
38  </body>
39 </html>

```

```

1 $(document).ready(function(){
2   /*Sélectionne tous les ancêtres de
l'élément qui possède l'id="enfants"
3   jusqu'au premier ancêtre possédant
une class="conteneur" sans l'inclure*/
4   $("#enfants").parentsUntil(".conteneur
").css("border", "1px solid black");
5 });

```

éléme

29

Cours jQuery

Un paragraphe HTML

Méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Méthodes permettant de descendre le DOM :

- children()
- find()

- Elément de pied de page n°1
- Elément de pied de page n°2

- Finalement, la méthode `closest()` va nous permettre de sélectionner le premier ancêtre correspondant au sélecteur qu'on va lui passer en argument.
- Attention : cette méthode commence sa recherche à partir de l'élément ou des éléments de la sélection initiale. Cela signifie que si ces éléments correspondent au sélecteur passé en argument ils seront sélectionnés.

ou

éléme

31

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <div>
11        <p>Un paragraphe HTML</p>
12      </div>
13      <div>
14        <p>Méthodes jQuery permettant de <span>remonter</span> le
DOM :</p>
15        <ul id="parents" class="bleu">
16          <li class="li1">parent()</li>
17          <li>parents()</li>
18          <li>parentsUntil()</li>
19          <li>closest()</li>
20        </ul>
21      </div>
22      <p>Méthodes permettant de <span>descendre</span> le DOM :
</p>
23      <ul id="enfants" class="bleu">
24        <li class="li1 bleu">children()</li>
25        <li>find()</li>
26      </ul>
27    </div>
28  </div>
29 </body>
30 </html>

```

```

1 $(document).ready(function(){
2   /*Sélectionne l'ancêtre div
le plus proche de l'élément
possédant
3     l'id="parents"*/
4
5   $("#parents").closest("div").cs
s("border", "1px solid black");
6
7   /*Sélectionne l'ancêtre
possédant une class="bleu" le
plus proche
8     des éléments possédant une
class="li1"*/
9   $(".li1").closest(".bleu").css(
"color", "blue");
10 });

```

Cours jQuery

Un paragraphe HTML

Méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Méthodes permettant de descendre le DOM :

- children()
- find()

Accéder aux enfants ou descendants d'un élément en jQuery

32

- Les méthodes `children()` et `find()` vont nous permettre d'accéder aux enfants et descendants d'un élément ou d'une collection d'éléments précédemment sélectionnés en jQuery.
- La méthode `children()` va nous permettre d'accéder aux éléments enfants (descendants directs) d'un ou d'une collection d'éléments HTML. On va pouvoir lui passer un sélecteur en argument qui va servir de filtre pour nous permettre de ne sélectionner que les enfants correspondant au filtre.
- La méthode `find()` va nous permettre de sélectionner tous les descendants (enfants, petits-enfants, etc.) d'un élément ou d'une collection d'éléments. Il va falloir passer un sélecteur en argument de cette méthode qui va servir de filtre pour nous permettre de ne sélectionner que les descendants correspondant au filtre.
- Une astuce ici : pour cibler tous les descendants d'un élément en utilisant `find()`, il va suffire de lui passer le sélecteur universel `*` en argument.

HTML

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6     <style>
7       .conteneur > *{margin-bottom: 5px;}
8     </style>
9   </head>
10  <body>
11    <div class="conteneur">
12      <h1>Cours jQuery</h1>
13      <div id="methodes">
14        <p>Méthodes jQuery permettant de <span>remonter</span> le
DOM :</p>
15        <ul id="parents">
16          <li class="bglb">parent()</li>
17          <li>parents()</li>
18          <li>parentsUntil()</li>
19          <li>closest()</li>
20        </ul>
21
22        <p>Méthodes permettant de <span>descendre</span> le DOM :
</p>
23        <ul id="enfants" class="bglb">
24          <li>children()</li>
25          <li>find()</li>
26        </ul>
27      </div>
28    </div>
29  </body>
30 </html>

```

JS

```

1 $(document).ready(function(){
2   /*Sélectionne tous les enfants
   (descendants directs) des éléments
   possédant
3     un attribut class="conteneur"*/
4
5   $(".conteneur").children().css("border"
6     , "1px solid black");
7
8   /*Sélectionne uniquement les
   éléments h1 descendants directs des
   éléments
9     possédant un attribut
   class="conteneur"*/
10  $(".conteneur").children("h1").css("col
11    or", "blue");
12
13  /*Cible tous les descendants de
   l'élément possédant l'id="methodes" qui
   possèdent un attribut class="bglb"*/
14  $("#methodes").find(".bglb").css("backg
15    round-color", "lightBlue");
16 });

```

Cours jQuery

Méthodes jQuery permettant de remonter le DOM :

- parent()
- parents()
- parentsUntil()
- closest()

Méthodes permettant de descendre le DOM :

- children()
- find()

Accéder aux éléments frères d'un élément en jQuery

34

- jQuery met également à notre disposition plusieurs méthodes nous permettant de sélectionner les éléments frères d'un élément ou d'une collection d'éléments. Deux éléments sont "frères" ("siblings" en anglais) si ils partagent le même parent (le même ancêtre direct).
- Nous allons ici étudier les méthodes `siblings()`, `next()`, `nextAll()`, `nextUntil()`, `prev()`, `prevAll()` et `prevUntil()`.
- La méthode `siblings()`, tout d'abord, va retourner l'ensemble de éléments frères d'un premier élément sélectionné ou d'un premier ensemble d'éléments sélectionnés. On va pouvoir passer un sélecteur qui agira comme filtre et nous permettra de ne récupérer que les éléments frères correspondant à celui-ci en argument de cette méthode.

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <div>
11        <p>Plus de méthodes pour traverser le DOM :</p>
12        <ul>
13          <li>siblings()</li>
14          <li>next()</li>
15          <li>nextAll()</li>
16          <li>nextUntil()</li>
17          <li>prev()</li>
18          <li>prevAll()</li>
19          <li>prevUntil()</li>
20        </ul>
21      </div>
22      <p>
23        Ces méthodes servent à accéder aux éléments frères d'une
24        première collection d'éléments.
25      </p>
26    </div>
27  </body>
28 </html>

```

```

1 $(document).ready(function(){
2   /*Sélectionne les éléments li
   qui sont les premiers enfants de
   leur parent
3   puis récupère tous leurs
   frères et leur applique une
   couleur de texte bleue*/
4   $("li:first-
   child").siblings().css("color",
   "blue");
5
6   /*Sélectionne les éléments li
   qui sont les premiers enfants de
   leur parent et
7   ne récupère que les éléments
   frères qui occupent une place
   impaire en comptant à partir de
   li:first-child*/
8   $("li:first-
   child").siblings(":odd").css("text-
   decoration", "underline");
9 });

```

Cours jQuery

Plus de méthodes pour traverser le DOM :

- siblings()
- next()
- nextAll()
- nextUntil()
- prev()
- prevAll()
- prevUntil()

Ces méthodes servent à accéder aux éléments frères d'une première collection d'éléments.

Manipuler le DOM avec les méthodes de type getter et setter jQuery

36

- Dans la plupart des langages informatiques, les fonctions / méthodes qui permettent d'obtenir ou de définir des informations fonctionnent par paire qu'on appelle "getters" et "setters".
- Le rôle d'un getter est de récupérer une information. Le rôle d'un setter est de définir ou de mettre à jour une information.
- Par convention, les fonctions de type getter commencent par le préfixe "get" tandis que les fonctions de type setter commencent par le préfixe "set". Cela nous permet de rapidement les identifier.
- Par exemple, le Javascript possède une méthode `getAttribute()` qui nous permet de récupérer un attribut HTML et une méthode `setAttribute()` qui nous permet de définir ou de mettre à jour la valeur d'un attribut pour un élément HTML.
- Il convient de noter que dans le cas où on sélectionne plusieurs éléments différents d'un coup, tous les éléments de la sélection seront affectés si on utilise l'une de ces méthodes en tant que setter mais seules les informations relatives au premier élément de la sélection seront renvoyées si on les utilise comme getter, à l'exception notable de la méthode `text()` qui va récupérer les valeurs de tous les éléments de la sélection.

Manipuler le DOM avec les méthodes de type getter et setter jQuery

37

- Voici la liste de ces méthodes (que nous allons étudier par la suite) :
- `html()` : Permet d'obtenir le contenu HTML du premier élément d'une sélection ou de définir le contenu HTML de tous les éléments de la sélection ;
- `text()` : Permet d'obtenir ou de définir les contenus textuels de tous les éléments d'une sélection ;
- `attr()` : Permet d'obtenir la valeur d'un attribut du premier élément d'une sélection ou de définir un ou plusieurs attributs pour tous les éléments de la sélection ;
- `prop()` : Permet d'obtenir la valeur d'une propriété du premier élément d'une sélection ou de définir une ou plusieurs propriétés pour tous les éléments de la sélection ;
- `val()` : Permet d'obtenir la valeur (le contenu de l'attribut `value`) du premier élément d'une sélection ou de définir une valeur pour tous les éléments de la sélection ;
- `css()` : Permet d'obtenir la valeur d'une propriété CSS pour le premier élément d'une sélection ou de définir les valeurs d'une ou de plusieurs propriétés CSS pour tous les éléments de la sélection ;
- `height()` : Permet d'obtenir la hauteur de la boîte élément du premier élément d'une sélection ou de définir cette hauteur pour tous les éléments de la sélection ;
- `innerHeight()` : Idem `height()` mais inclut les marges intérieures (`padding`) dans le calcul ;
- `outerHeight()` : Idem `height()` mais inclut les marges intérieures (`padding`), les bordures et de manière optionnelle les marges extérieures (`margin`) dans le calcul ;
- `width()` : Permet d'obtenir la largeur de la boîte élément du premier élément d'une sélection ou de définir cette largeur pour tous les éléments de la sélection ;
- `innerWidth()` : Idem `width()` mais inclut les marges intérieures (`padding`) dans le calcul ;
- `outerWidth()` : Idem `width()` mais inclut les marges intérieures (`padding`), les bordures et de manière optionnelle les marges extérieures (`margin`) dans le calcul ;
- `offset()` : Permet d'obtenir les coordonnées actuelles du premier élément d'une sélection ou de définir les coordonnées de tous les éléments de la sélection relativement au document ;
- `scrollLeft()` : Permet d'obtenir la position actuelle de la barre de défilement horizontale du premier élément d'une sélection ou de définir cette position tous les éléments de la sélection ;
- `scrollTop()` : Permet d'obtenir la position actuelle de la barre de défilement verticale du premier élément d'une sélection ou de définir cette position tous les éléments de la sélection.

La méthode jQuery html()

- La méthode `html()` va retourner le contenu HTML du premier élément d'une sélection si on l'utilise sans argument ou définir le contenu HTML de tous les éléments d'une sélection si on précise ce contenu en argument.

La méthode jQuery html()

- On commence ici par cibler nos éléments body et le paragraphe possédant l'id="p1" et on utilise la méthode html() sans argument sur les deux objets jQuery renvoyés qui représentent notre élément body et notre élément p.
- La méthode html() agit ici comme un getter et va récupérer le contenu HTML du corps de page et de notre premier paragraphe. On place les résultats renvoyés par html() dans deux variables let page et let p1 et on affiche leur contenu dans la console JavaScript avec l'instruction console.log() pour s'assurer de ce qu'elle contiennent.
- Note : pour afficher la console, vous devez activer les outils pour développeurs de votre navigateur puis effectuer un clic droit sur la page et choisir "inspecter".
- On cible ensuite l'élément possédant l'attribut id="p2" et on utilise notre méthode html() en lui passant cette fois-ci des données en argument. La méthode html() va alors agir comme un setter et modifier le contenu HTML de notre deuxième paragraphe.

HTML

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <p id="p1">Un paragraphe écrit en HTML</p>
11      <p id="p2">Un deuxième paragraphe</p>
12    </div>
13  </body>
14 </html>

```

JS

```

1 $(document).ready(function(){
2   //On utilise html() comme getter
3   let page = $("body").html();
4   let p1 = $("#p1").html();
5
6   //On affiche le contenu de nos variables dans la console
7   console.log("Page : " + page);
8   console.log("p1 : " + p1);
9
10  //On utilise html() comme setter
11  $("#p2").html("<strong>Nouveau contenu !</strong>");
12 });

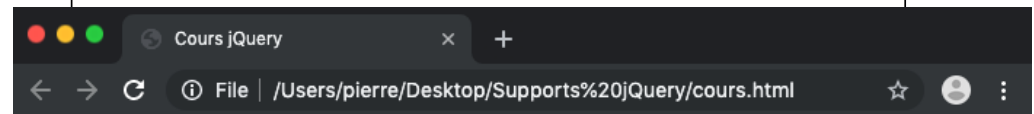
```

La méthode jQuery html()

- On commence ici par cibler nos éléments body et le paragraphe possédant l'id="p1" et on utilise la méthode html() sans argument sur les deux objets jQuery renvoyés qui représentent notre élément body et notre élément p.
- La méthode html() agit ici comme un getter et va récupérer le contenu HTML du corps de page et de notre premier paragraphe. On place les résultats renvoyés par html() dans deux variables let page et let p1 et on affiche leur contenu dans la console JavaScript avec l'instruction console.log() pour s'assurer de ce qu'elle contiennent.
- Note : pour afficher la console, vous devez activer les outils pour développeurs de votre navigateur puis effectuer un clic droit sur la page et choisir "inspecter".
- On cible ensuite l'élément possédant l'attribut id="p2" et on utilise notre méthode html() en lui passant cette fois-ci des données en argument. La méthode html() va alors agir comme un setter et modifier le contenu HTML de notre deuxième paragraphe.

HTML

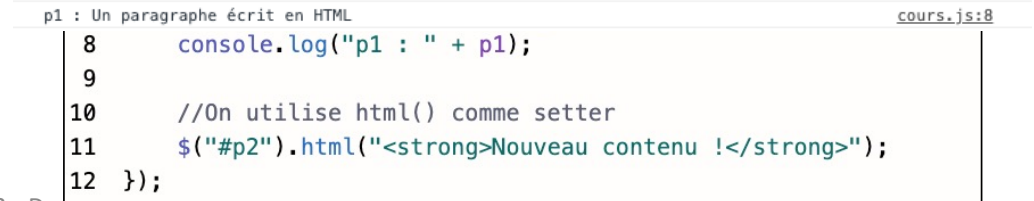
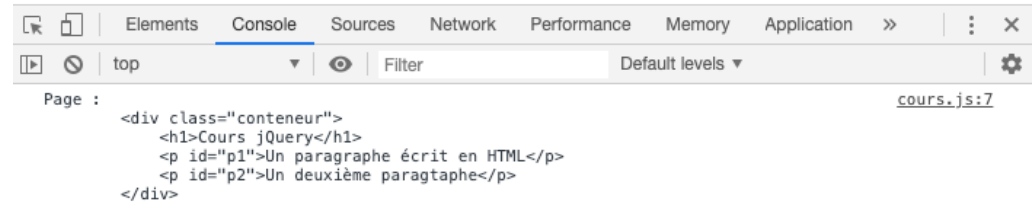
```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
```



Cours jQuery

Un paragraphe écrit en HTML

Nouveau contenu !



La méthode jQuery html()

- La méthode text() va nous permettre de récupérer ou de modifier le contenu textuel de tous les éléments d'une sélection.
- A la différence de la méthode html(), seul le contenu textuel va pouvoir être récupéré et on ne va pouvoir également définir que du contenu textuel dans nos éléments.
- Comme vous pouvez le voir dans cet exemple, la méthode text() va bien récupérer le contenu textuel (et uniquement le contenu textuel) de tous les éléments d'une sélection lorsqu'on l'utilise en tant que getter. C'est un cas unique.
- Lorsqu'on utilise text() en tant que setter, cette méthode ne va pouvoir définir qu'un nouveau contenu textuel. Si on tente de lui passer du balisage HTML comme on l'a fait dans l'exemple ci-dessus, les balises vont être neutralisées et s'afficher sous format texte.

HTML

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <p id="p1">Un paragraphe écrit en HTML</p>
11      <p id="p2">Un <span>deuxième</span> paragraphe</p>
12    </div>
13  </body>
14 </html>

```

JS

```

1 $(document).ready(function(){
2   //On utilise text() comme getter
3   let texteP = $("p").text();
4
5   //On affiche le contenu de notre variable dans la console
6   console.log("Contenu textuel des paragraphes : " + texteP);
7
8   //On utilise text() comme setter
9   $("#p2").text("<strong>Nouveau contenu !</strong>");
10 });

```

La méthode jQuery html()

- La méthode text() va nous permettre de récupérer ou de modifier le contenu textuel de tous les éléments.
- A la différence de la méthode html(), on ne va pas pouvoir récupérer le contenu textuel d'un élément.
- Comme vous l'avez vu, la méthode html() permet de récupérer le contenu textuel d'un élément (ou de le modifier) de toute la page, mais aussi d'un seul élément, lorsque l'on l'utilise avec un sélecteur unique.
- Lorsqu'on utilise cette méthode pour passer du contenu HTML à un élément, les balises HTML sont neutralisées et s'affichent sous format texte.

```
HTML
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
```

Cours jQuery

Un paragraphe écrit en HTML

Nouveau contenu !

Elements Console Sources Network Performance Memory Application >> | : X

top Filter Default levels

Contenu textuel des paragraphes : Un paragraphe écrit en HTMLUn deuxième paragraphe cours.js:6

```
8 //On utilise text() comme setter
9 $("#p2").text("<strong>Nouveau contenu !</strong>");
10 };
```

Récupérer la valeur d'un attribut ou définir un attribut

43

- On va pouvoir utiliser la méthode `attr()` pour récupérer la valeur d'un attribut, ajouter un attribut ou modifier la valeur d'un attribut déjà existant.
- Pour récupérer la valeur d'un attribut, nous allons utiliser `attr()` en lui passant le nom de l'attribut dont on souhaite récupérer la valeur en argument. Cette méthode va alors agir comme un getter et retourner la valeur de l'attribut du premier élément de la notre sélection uniquement.
- Pour mettre à jour la valeur d'un attribut ou pour définir un nouvel attribut, nous allons passer l'attribut en question en premier argument de `attr()` et sa (nouvelle) valeur associée en second argument. Si l'attribut existait déjà, sa valeur sera mise à jour. Si il n'existait pas, il sera ajouté à chaque élément de notre sélection.

```
HTML
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <p id="p1">Un paragraphe écrit en HTML</p>
11      <p id="p2">Un autre paragraphe</p>
12    </div>
13    <p>Un paragraphe en dehors du div</p>
14    <p>Un <a href="https://www.pierre-giraud.com">lien</a> vers mon
      site</p>
15    <span id="valAttr"></span>
16  </body>
17 </html>
```

```
JS
1 $(document).ready(function(){
2   //Retourne la valeur du premier id d'un paragraphe trouvé
3   let pid = $("p").attr("id");
4
5   //Affiche la valeur de l'id récupéré dans notre span id="valAttr"
6   document.getElementById("valAttr").textContent = "Valeur du 1er id de
  p : " + pid;
7
8   //Ajoute un attribut target à notre lien
9   $("a").attr("target","_blank");
10 });
```

Récupérer la valeur d'un attribut ou définir un attribut

44

- On va pouvoir utiliser la méthode `attr()` pour récupérer la valeur d'un attribut, ajouter un attribut ou modifier la valeur d'un attribut déjà existant.
- Pour récupérer la valeur d'un attribut, nous allons utiliser `attr()` en lui passant le nom de l'attribut dont on souhaite récupérer la valeur en argument. Cette méthode va alors agir comme un getter et retourner la valeur de l'attribut du premier élément de la notre sélection uniquement.
- Pour mettre à jour la valeur d'un attribut ou pour définir un nouvel attribut, nous allons passer l'attribut en question en premier argument de `attr()` et sa (nouvelle) valeur associée en second argument. Si l'attribut existait déjà, sa valeur sera mise à jour. Si il n'existait pas, il sera ajouté à chaque élément de notre sélection.

```
HTML
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5   </head>
6   <body>
7     <h1>Cours jQuery</h1>
8     <p>Un paragraphe écrit en HTML</p>
9     <p>Un autre paragraphe</p>
10    <a href="#">Un lien vers mon site</a>
11  </body>
12 </html>
```

```
JS
1 $(document).ready(function() {
2   //Récupère le premier lien
3   let a = $("a").first();
4
5   //Ajoute un attribut target à notre lien
6   a.attr("target", "_blank");
7
8   //Ajoute un attribut target à notre lien
9   $("a").attr("target", "_blank");
10 });
```

Un lien vers mon site

Valeur du 1er id de p : p1

"valAttr" du 1er id de p

La méthode `prop()` et les propriétés du DOM

- La méthode `prop()` va nous permettre de récupérer la valeur d'une propriété du premier élément de notre sélection ou d'ajouter une propriété ou de modifier la valeur d'une propriété déjà existante pour chaque élément d'une sélection.
- Pour que vous compreniez l'intérêt de cette méthode, il convient de bien définir la différence entre un attribut et une propriété en HTML.
- Un attribut sert à fournir des informations supplémentaires par rapport à un élément et à son comportement. Un attribut est toujours défini dans notre fichier HTML de départ.

La méthode prop() et les propriétés du DOM

- Une fois qu'un fichier HTML est chargé dans un navigateur, ce dernier va créer un DOM qui est une représentation de notre fichier de départ.
 - A ce stade, en Javascript, notre fichier HTML va être vu sous la forme de noeuds ou "node" qui sont des objets. Les attributs sont alors représentés sous forme de propriétés de nos noeuds éléments.
- On a donc ici deux représentations qui coexistent : notre fichier HTML avec ses éléments et ses attributs d'un côté, et le DOM avec ses noeuds et ses propriétés. Ces deux représentations existent en même temps.
- L'idée ici est qu'on va ensuite pouvoir modifier le DOM, et donc modifier les différents objets et les valeurs des propriétés de celui-ci sans forcément modifier la valeur des attributs de notre fichier HTML.

La méthode prop() et les propriétés du DOM

```

HTML
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <p id="p1">Un paragraphe écrit en HTML</p>
11      <p id="p2">Un autre paragraphe</p>
12    </div>
13    <p>Un paragraphe en dehors du div</p>
14    <p>Un <a href="https://www.pierre-giraud.com">lien</a> vers mon
    site</p>
15    <span id="valAttr"></span>
16
17    <input id="check1" type="checkbox" checked="checked">
18    <label for="check1">Cochez / décochez moi</label>
19    <br>
20    <input id="check2" type="checkbox" checked="toto">
21    <label for="check2">Cochez / décochez moi</label>
22  </body>
23 </html>

```

Cours jQuery

Un paragraphe écrit en HTML

Un autre paragraphe

Un paragraphe en dehors du div

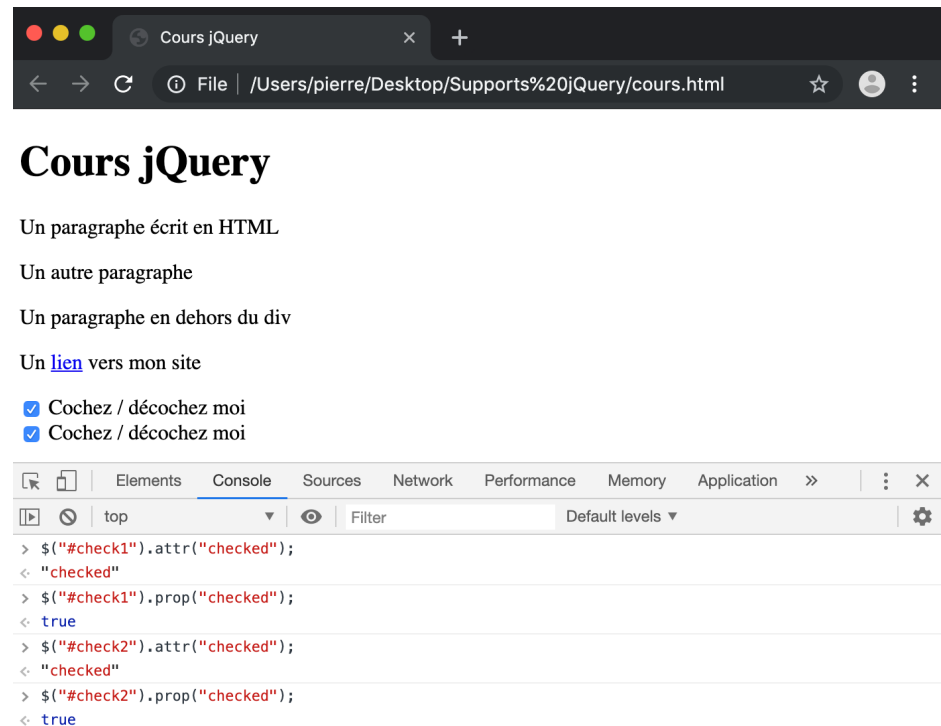
Un [lien](#) vers mon site

☒ Cochez / décochez moi

☒ Cochez / décochez moi

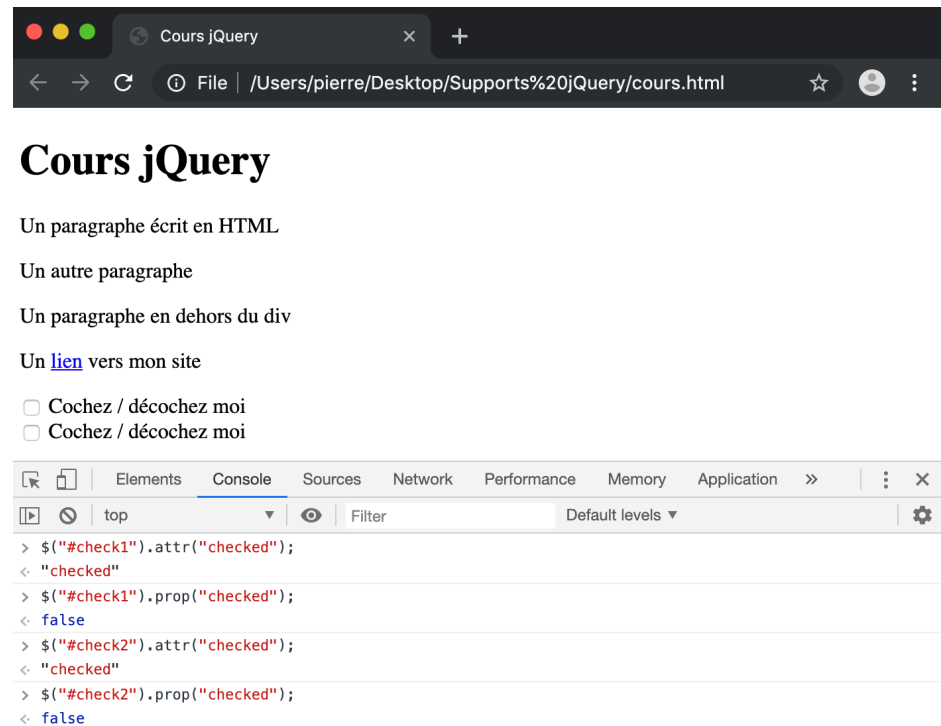
La méthode prop() et les propriétés du DOM

- Selon la définition du W3C, l'attribut checked est un attribut booléen, ce qui signifie qu'il n'a que deux états associés possibles : true (case cochée) et false (case décochée).
- Dès qu'on mentionne cet attribut dans notre fichier HTML, et cela quelque soit la valeur qu'on lui donne (et même si on ne lui donne pas de valeur), sa valeur de résolution dans le DOM sera définie à true. Cela signifie que la valeur de la propriété associée va être true.
- On a donc deux valeurs différentes pour nos attribut checked définis dans notre fichier HTML et les propriétés checked associées.



La méthode prop() et les propriétés du DOM

- La valeur des attributs checked n'est pas modifiée. En revanche, la valeur des propriétés checked associée va être modifiée : celle-ci va devenir false.
- Cela est dû au fait qu'une case à cocher est un élément "dynamique" c'est-à-dire un élément avec lequel l'utilisateur va pouvoir interagir.
- Pour tous ces éléments dynamiques, vous pouvez donc considérer que la valeur de l'attribut HTML va être celle définie initialement dans le fichier HTML tandis que la valeur de la propriété associée va pouvoir être modifiée dynamiquement. La propriété prop() va donc aller récupérer dans le DOM la dernière valeur connue de cette propriété.



The screenshot shows a web browser window titled "Cours jQuery" with the address bar displaying the file path: `File | /Users/pierre/Desktop/Supports%20jQuery/cours.html`. The page content includes:

- Un paragraphe écrit en HTML
- Un autre paragraphe
- Un paragraphe en dehors du div
- Un [lien](#) vers mon site
- ☐ Cochez / décochez moi
- ☐ Cochez / décochez moi

The browser's developer console is open, showing the following JavaScript commands and their results:

```
> $("#check1").attr("checked");
< "checked"
> $("#check1").prop("checked");
< false
> $("#check2").attr("checked");
< "checked"
> $("#check2").prop("checked");
< false
```

Supprimer un attribut ou une propriété

- Pour supprimer un attribut, on va pouvoir utiliser la méthode `removeAttr()` en lui passant en argument le nom de l'attribut qu'on souhaite supprimer.
- De la même façon, on utilisera `removeProp()` pour supprimer une propriété d'une sélection d'éléments.

HTML

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours jQuery</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <div class="conteneur">
9       <h1>Cours jQuery</h1>
10      <p id="p1">Un paragraphe écrit en HTML</p>
11      <p id="p2">Un autre paragraphe</p>
12    </div>
13    <p>Un paragraphe en dehors du div</p>
14    <p>Un <a href="https://www.pierre-giraud.com">lien</a> vers mon
      site</p>
15  </body>
16 </html>

```

JS

```

1 $(document).ready(function(){
2   $("a").removeAttr("href");
3 });

```

Cours jQuery

Un paragraphe écrit en HTML

Un autre paragraphe

Un paragraphe en dehors du div

Un lien vers mon site